

Workshop on Computational Fluid Dynamics (CFD)
with OpenFOAM
IMP-PAN Gdansk

Tutorial #1: OpenFOAM Framework

Incompressible Transient Flow: pitzDaily with pimpleFoam

Robert Castilla
CATMech - Universitat Politècnica de Catalunya
Department of Fluid Mechanics

2025-26
Version 1.0

This tutorial introduces the fundamental workflow of Computational Fluid Dynamics (CFD) using OpenFOAM in a terminal environment. Through the classic **pitzDaily** case with the **pimpleFoam** solver, students will learn to set up, run, and analyze an incompressible transient flow simulation. No graphical interface is required—all operations are performed via command line, making this ideal for remote servers and building core CFD skills.

Contents

1. Introduction and Theoretical Background	3
1.1. What is pitzDaily?	3
1.2. Governing Equations	4
1.3. Case Overview	5
2. Terminal Environment Setup	5
2.1. Accessing OpenFOAM	5
2.2. Navigating to Tutorials	5
3. Case Structure Exploration	6
3.1. Understanding OpenFOAM Directory Structure	6
3.2. Examining Key Files	6

4. Running the Simulation	7
4.1. Pre-processing	7
4.2. Running pimpleFoam	8
4.3. Monitoring Convergence	8
5. Post-Processing in Terminal	9
5.1. Basic Results Examination	9
5.2. Using OpenFOAM Command Line Tools	9
5.3. Plotting residuals	11
5.3.1. Exercise with runtime post-processing	11
6. Basic Visualization in paraview	13
7. Common Issues and Troubleshooting	14
7.1. Error: "command not found"	14
7.2. Error: "cannot find blockMeshDict"	14
7.3. Simulation Diverges (Residuals Grow)	15
8. Student Exercises and Extensions	15
8.1. Basic Exercises (Required)	15
8.2. Advanced Exercises (Optional)	15
9. Deliverables and Assessment	15
9.1. Required Files	16
9.2. Report Questions	16
9.3. Submission Format	16
A. Appendix: Useful Terminal Commands Reference	17
A.1. OpenFOAM-Specific	17
A.2. General Linux Commands	17
B. Quick Reference: File Locations in pitzDaily	17

Learning Objectives

By the end of this tutorial the student will be able to

1. **Navigate** OpenFOAM case directory structure
2. **Understand** basic OpenFOAM file formats
3. **Run** a simple transient simulation
4. **Analyze** basic results in terminal

1. Introduction and Theoretical Background

1.1. What is pitzDaily?

The **pitzDaily** case simulates 2D steady, turbulent flow in a backward-facing step configuration. It's included in the OpenFOAM tutorials and serves as an excellent starting point due to its:

- Simple geometry but interesting physics (recirculation zone)
- Complete setup with all necessary files
- Quick runtime (minutes on most computers)

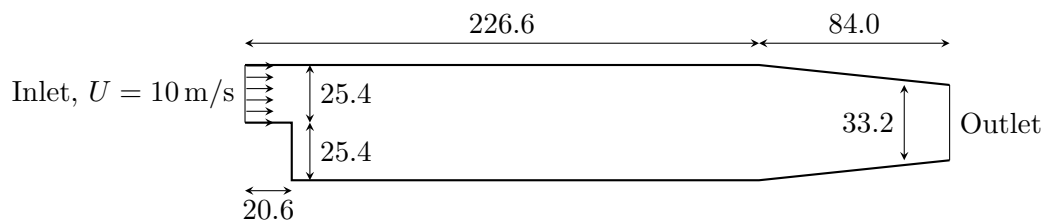


Figure 1: pitzDaily geometry. Lengths are in millimeter.

More information can be found on the [OpenFOAM website](#).

1.2. Governing Equations

The **pimpleFoam** solver implements the incompressible Reynolds-Averaged Navier-Stokes (RANS) equations:

$$\nabla \cdot \mathbf{U} = 0 \quad (1)$$

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) = -\nabla \left(\frac{p}{\rho} \right) + \nabla \cdot (\nu_{\text{eff}} \nabla \mathbf{U}) \quad (2)$$

where $\nu_{\text{eff}} = \nu + \nu_t$ is the effective viscosity (molecular + turbulence viscosity).

Note that, since the fluid is incompressible, instead of pressure p , this equation deals with kinematic pressure, $\frac{p}{\rho}$, that has units of m^2/s^2 .

IMPORTANT: The solver **pimpleFoam** keeps the name p for the kinematic pressure.

1.3. Case Overview

Parameter	Value
Solver	pimpleFoam (transient, incompressible)
Turbulence model	$k-\epsilon$ (standard)
Flow regime	Turbulent
Geometry	2D backward-facing step
Reynolds number	$\sim 2 \times 10^4$ (based on step height)
Expected runtime	20-40 seconds

Table 1: pitzDaily case specifications

2. Terminal Environment Setup

2.1. Accessing OpenFOAM

Open your terminal and verify OpenFOAM is available:

```
foamVersion
```

IMPORTANT: If you see "command not found," you may need to source the OpenFOAM environment first:

```
# For system-wide installation
source /usr/lib/openfoam/openfoam-2312/etc/bashrc
# For a local bash session with OpenFOAM
openfoam2512
```

2.2. Navigating to Tutorials

OpenFOAM includes a comprehensive set of tutorials. Navigate to the incompressible tutorials:

```
# Go to OpenFOAM tutorials directory
cd $FOAM_TUTORIALS

# List available tutorials
ls

# Navigate to incompressible pimpleFoam tutorials
cd incompressible/pimpleFoam
```

3. Case Structure Exploration

3.1. Understanding OpenFOAM Directory Structure

Copy the pitzDaily case in a local folder

```
# Make a local folder
mkdir -p $HOME/Tutorials/Tutorial_1

# Copy the pitzDaily tutorial in this folder
cp -r ./pitzDaily $HOME/Tutorials/Tutorial_1

# Go to the tutorial place
cd $HOME/Tutorials/Tutorial_1
```

IMPORTANT: Never play with a tutorial in the OpenFOAM distribution home folder. Always make a copy. In this way you will have always a "clean" copy.

Examine the pitzDaily case structure:

```
# Go to the pitzDaily case
cd pitzDaily

# List all contents with details
ls -la

# See the directory structure
tree -L 2
```

IMPORTANT: Every OpenFOAM case has this standard structure:

```
pitzDaily/
|-- 0/          # Initial and boundary conditions
|-- constant/   # Physical properties and mesh
|   |-- polyMesh/ # Mesh files
|   `-- transportProperties
|-- system/     # Solution control
|   |-- controlDict
|   |-- fvSchemes
|   `-- fvSolution
```

3.2. Examining Key Files

Let's look at the most important configuration files:

Step 1: Initial Conditions (0/ directory):

```
# View velocity boundary conditions
cat 0/U

# View pressure boundary conditions
cat 0/p

# View turbulence variables
cat 0/k
cat 0/epsilon
```

Step 2: Physical Properties:

```
# Check fluid properties
cat constant/transportProperties
```

This shows:

```
transportModel  Newtonian;

nu              1e-05;
```

Step 3: Solution Control:

```
# Check solver settings
cat system/fvSolution

# Check discretization schemes
cat system/fvSchemes

# Check runtime control
cat system/controlDict
```

TIP: Use `less` or `more` for longer files: `less system/controlDict`

4. Running the Simulation

4.1. Pre-processing

Before running, we need to generate the mesh:

```
# Generate the mesh using blockMesh, with the foamJob utility
foamJob -screen -log-app blockMesh

# Check mesh quality
foamJob -screen -log-app checkMesh
```

WARNING: If `blockMesh` fails, check for error messages. Common issues include missing dictionaries or syntax errors in `system/blockMeshDict`.

The utility `foamJob` creates a log file.

```
# Check the content of the log files
less log.blockMesh
less log.checkMesh
```

4.2. Running pimpleFoam

Now run the simulation:

```
# Run the solver
foamJob -s -log-app pimpleFoam
```

4.3. Monitoring Convergence

Monitor convergence:

```
# Watch residuals in real-time (if running in foreground)
# OR check the log file periodically:

# Show last 20 lines of residuals
tail -20 log.pimpleFoam | grep "Solving for"

# Check final residuals
tail -50 log.pimpleFoam | grep "Final residual"
```

Plot it using `gnuplot`

```
# Generate residual files from the log file
foamLog log.pimpleFoam

# Visualize the residuals of the main variables with gnuplot
gnuplot
gnuplot> set logscale y
gnuplot> plot 'logs/p_0' with lines, 'logs/Ux_0' with lines, 'logs/Uy_0' with
lines, 'logs/epsilon_0' with lines, 'logs/k_0' with lines
```

You will get a picture similar to that in Figure 2.

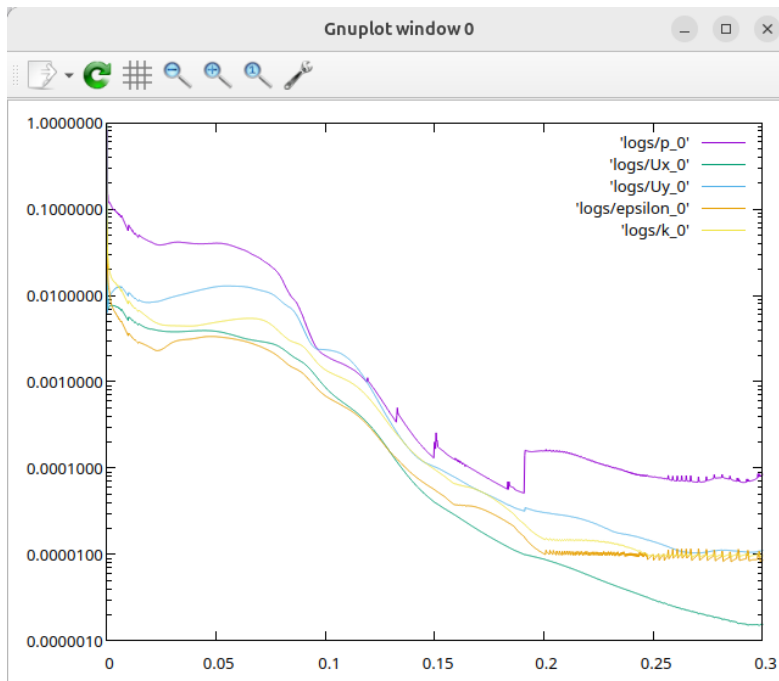


Figure 2: Plot of residuals

5. Post-Processing in Terminal

5.1. Basic Results Examination

OpenFOAM stores results in time directories. Examine the final results:

```
# List time directories
foamListTimes

# Go to final time directory (0.3 for this simulation)
cd 0.3

# List available fields
ls
```

5.2. Using OpenFOAM Command Line Tools

- Check field statistics

```
# Go back to case root if you are still in the time folder
cd ..
```

```
# Get some statistics
postProcess -func writeCellCentres
postProcess -func minMaxMagnitude
```

- Check average pressure in inlet:

```
# Get the dictionary
foamGetDict patchAverage
```

This will add the file **system/patchAverage** to the case. The name of this function can be changed, and it can be edited to define the patch and the variable to be averaged

```
# Rename the function
mv system/patchAverage system/inletAveragePressure
# Edit the function (you can use vim, nano, emacs, ...)
vim system/inletAveragePressure
```

```
...
name      inlet;
fields    (p);

operation areaAverage;
#includeEtc "caseDicts/postProcessing/surfaceFieldValue/
    patch.cfg"
...
```

```
# compute average pressure in inlet
postProcess -func inletAveragePressure
```

TIP: The command **postProcess -list** will list all the functions available.

Besides the screen, the information from **postProcess** is also saved in the **postProcessing** folder. In order to plot the results of a post-processing function (pressure average in inlet patch, for instance), the utility **foamMonitor** is used

```
foamMonitor postProcessing/inletAveragePressure/0.3/surfaceFieldValue.dat
```

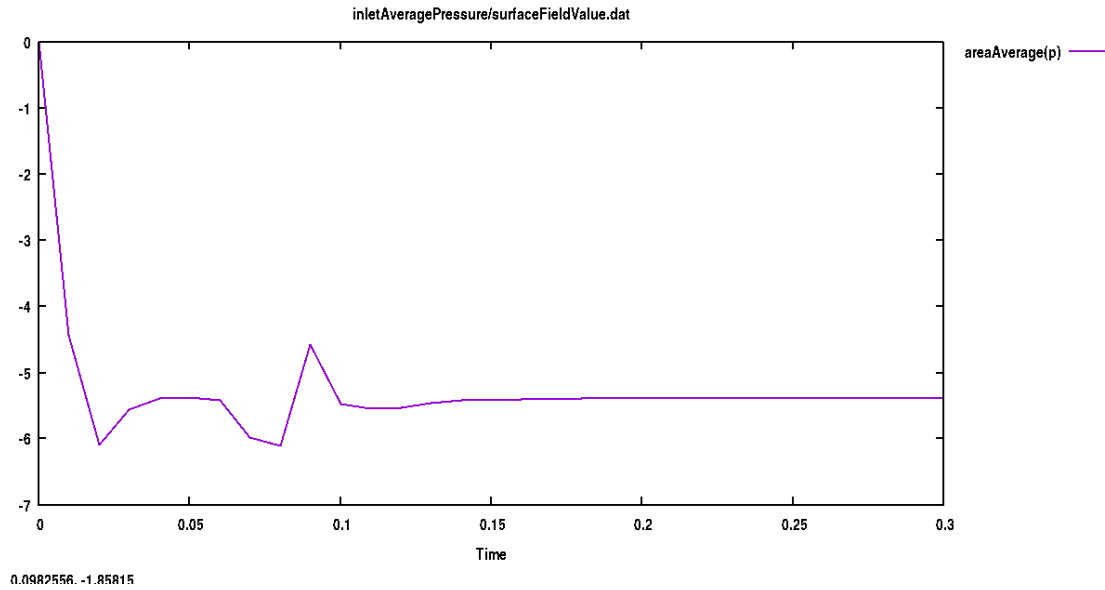


Figure 3

5.3. Plotting residuals

In the last section we have seen how to monitor residuals from the log file. But it is also possible to save the residuals in a data-file in order to post-process them with `foamMonitor` or any other application (a Python script, for instance)

In order to log the residuals in run time, the function has to be added in the `system` folder with

```
foamGetDict solverInfo
```

It will give two options. Usually it is better the first one.

5.3.1. Exercise with runtime post-processing

Let's run again the previous simulation, but this time with computation, in run-time, of

1. Residuals
2. Total pressure
3. Average of total pressure in inlet and outlet
4. Vorticity and second invariant of velocity gradient tensor, Q

Go back to the case root and clean it

```
foamListTimes -rm
```

Get the required dictionaries

```
# Get the function to record residuals
foamGetDict solverInfo
# Get the function to compute total pressure
foamGetDict totalPressureIncompressible
# Get the functions to compute total pressure in inlet and outlet
foamGetDict patchAverage
# move this file to a personalized function for inlet patch
mv system/patchAverage system/inletTotalPressureAverage
# copy this file for the outlet patch
cp system/inletTotalPressureAverage system/outletTotalPressureAverage
# Get the functions to compute vorticity and Q
foamGet vorticity
foamGet Q
```

Edit the function files to fit the simulation requirements.

The **system/solverInfo** should look like that:

```
#includeEtc "caseDicts/postProcessing/numerical/solverInfo.cfg"

fields (p U k omega);
```

The **system/totalPressureIncompressible** is like that:

```
#includeEtc "caseDicts/postProcessing/pressure/
    totalPressureIncompressible.cfg"

pRef      0.0;
rhoInf    1.2;

executeControl timeStep;
```

Change the density in function of the fluid. The keyword for **executeControl** is included in order to compute total pressure in every time step, to compute average values in the next functions. If this statement is not included, by default it is executed and written just in the **writeTime**, i.e. in the times when fields are written to disc. The **system/inletTotalPressureAverage** file is like that:

```
name      inlet;
fields    (total(p));

operation areaAverage;
#includeEtc "caseDicts/postProcessing/surfaceFieldValue/patch.cfg"
```

and the file `system/inletTotalPressureAverage` is the same, but with the `name` `outlet;` statement. The functions `system/vorticity` and `system/Q` have to be kept unmodified.

Finally, in order the solver to read all these functions, they have to be included in the `system/controlDict` file:

```
name      inlet;
fields    (total(p));

operation areaAverage;
#includeEtc "caseDicts/postProcessing/surfaceFieldValue/patch.
          cfg"
```

6. Basic Visualization in paraview

Usually the program `paraview` is used to visualize the results from OpenFOAM simulations. A wrapper `paraFoam` is available to call the program after performing some internal operations

```
# Run the paraFoam script
paraFoam
```

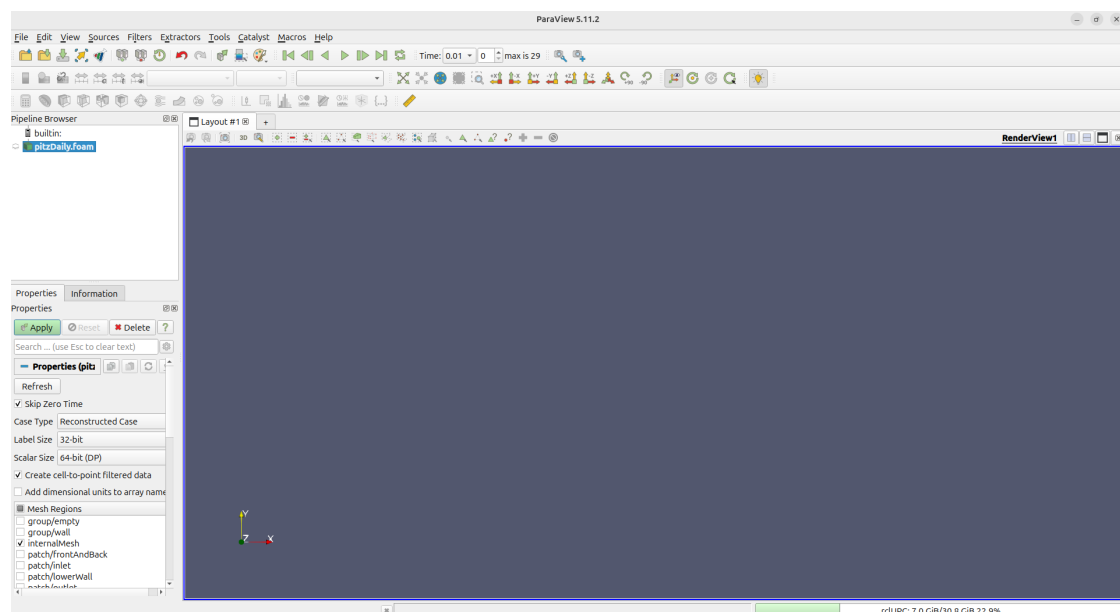


Figure 4: Start screen for paraview, called with `paraFoam`.

Click on the **Apply** button. By default, you will see the pressure distribution (Figure 5).

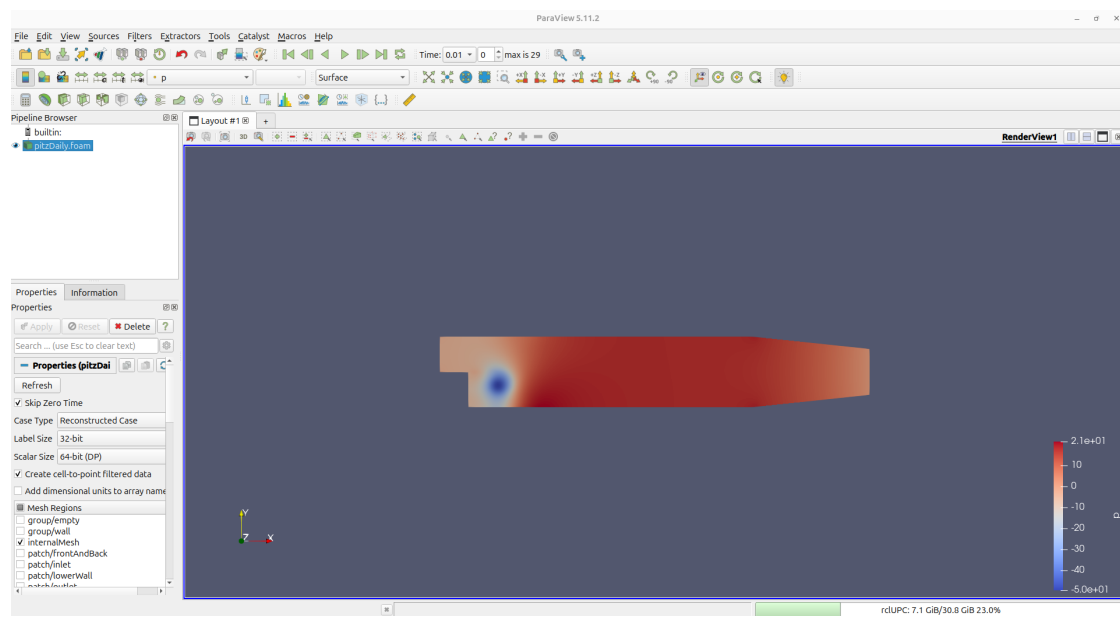


Figure 5: Pressure distribution of the flow.

7. Common Issues and Troubleshooting

7.1. Error: "command not found"

- **Cause:** OpenFOAM environment not sourced
- **Solution:**

```
source /path/to/OpenFOAM/etc/bashrc
```

7.2. Error: "cannot find blockMeshDict"

- **Cause:** Not in correct directory or file missing
- **Solution:**

```
# Check current directory
pwd

# List contents of system
ls system
```

7.3. Simulation Diverges (Residuals Grow)

- **Cause:** Poor initial conditions or solver settings
- **Solution:**
 1. Reduce under-relaxation factors in `system/fvSolution`
 2. Check boundary conditions are physically realistic
 3. Try running `potentialFoam` first to get better initial fields

8. Student Exercises and Extensions

8.1. Basic Exercises (Required)

1. **Boundary Condition Modification:**
 - Change inlet velocity from 10 m s^{-1} to 20 m s^{-1}
 - Update both `0/U` and `0/k/0/epsilon` appropriately
 - **Question:** How does Reynolds number affect recirculation zone size?
2. **Turbulence Model Comparison:**
 - Change from $k\text{-}\epsilon$ to $k\text{-}\omega$ SST in `constant/turbulenceProperties`
 - **Question:** Which model converges faster? Which gives higher k values?

8.2. Advanced Exercises (Optional)

1. **Mesh Refinement Study:**
 - Modify `system/blockMeshDict` to double cells in x-direction
 - Compare velocity profiles at outlet
2. **Custom Geometry:**
 - Create a simple 2D channel (modify `blockMeshDict`)
 - Reuse boundary conditions from `pitzDaily`
 - Compare pressure drop with analytical solution

9. Deliverables and Assessment

Submit the following to Moodle:

9.1. Required Files

1. Modified `system/controlDict` with your student ID in comments
2. Final `log.simpleFoam` file
3. Screen capture of terminal showing:
 - Successful `blockMesh` execution
 - Final residuals from `simpleFoam`
 - Output of `checkMesh`

9.2. Report Questions

Answer the following in a PDF report:

1. What are the final residuals for U_x , U_y , p , k , and ϵ ?
2. How many iterations were needed for convergence?
3. What is the maximum pressure in the domain? Minimum?
4. Based on the recirculation zone length, what is the reattachment length?
5. If you changed parameters in exercises, discuss how they affected results.

9.3. Submission Format

- **Filename:** StudentID_pitzDaily.zip
- **Contents:**

```
StudentID_pitzDaily/  
|-- report.pdf  
|-- controlDict_modified  
|-- log.simpleFoam  
|-- screenshots/  
|   |-- blockMesh.png  
|   |-- finalResiduals.png  
|   |-- checkMesh.png  
\\-- README.txt (any notes/issues encountered)
```


A. Appendix: Useful Terminal Commands Reference

A.1. OpenFOAM-Specific

Command	Purpose
<code>foamJob</code>	Run an openfoam command saving the log file
<code>blockMesh</code>	Generate block-structured mesh
<code>checkMesh</code>	Check mesh quality
<code>simpleFoam</code>	Run steady incompressible solver
<code>foamLog</code>	Parse log file for residuals
<code>postProcess</code>	Execute function objects
<code>paraFoam</code>	Launch visualization (if GUI available)
<code>reconstructPar</code>	Reconstruct parallel case

A.2. General Linux Commands

Command	Purpose
<code>grep "keyword" file</code>	Search for text in file
<code>tail -f log</code>	Follow log file in real-time
<code>head -n 20 file</code>	Show first 20 lines
<code>cd ~/</code>	Go to home directory
<code>pwd</code>	Print working directory
<code>ls -la</code>	List all files with details
<code>tree -L 2</code>	Show directory tree (2 levels)

B. Quick Reference: File Locations in pitzDaily

- Mesh definition: `constant/polyMesh/blockMeshDict`
- Boundary conditions: `0/U`, `0/p`, `0/k`, `0/epsilon`
- Solver settings: `system/fvSolution`
- Discretization schemes: `system/fvSchemes`
- Runtime control: `system/controlDict`
- Material properties: `constant/transportProperties`, `constant/turbulenceProperties`